

claude-windows / 985cc286-9e1e-4217-87c3-2d418bc4fa9c

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\985cc286-9e1e-4217-87c3-2d418bc4fa9c\subagents\workflows\wf_2ae09dfb-e3e\agent-a22756e0d5f45aef4.jsonl`
- SHA-256: `822ff33d9066b75c731b1a6eddadd34c3d0f312eb5d0e95344fb6494f347481`
- Source modified: `2026-06-13T19:56:55+00:00`
- Imported at: `2026-07-05T16:48:25+00:00`
- project: `wf_2ae09dfb-e3e`
- session_id: `985cc286-9e1e-4217-87c3-2d418bc4fa9c`
```

Transcript

1. user (2026-06-13T19:55:36.014Z)

Map the learned-LOVO surface for P3 (fusion learned head). Read backend/eval/distill_local.py (FEATURE_NAMES, build_candidates, predict_rallies, run, the LOVO loop), backend/eval/classifier.py (LogisticRegression), backend/eval/training.py (label_candidates), and backend/eval/experiment.py (Variant.feature_names, compare, evaluate_variant, load_video_candidates). Report EXACTLY how to measure shuttle-only vs shuttle+person LOVO F1 on the 6 golden videos: how to build a per-candidate labeled dataset carrying BOTH the 5 shuttle features AND the ~5 person features, and how to drive the harness compare()/LOVO to report the held-out lift. Identify the cleanest seam (extend distill_local? a new eval driver? feed VideoCandidates to experiment.compare?). Repo root = C:/Users/avidu/Projects/badminton-highlight-indexer. Read the ACTUAL files, quote file:line. The fusion P2 segmenter (FusionSegmenter), fusion_features.py, and PersonDetector.detections_per_frame are ALREADY on master. The experiment harness (backend/eval/experiment.py: Variant/compare/compare_unlabeled/load_video_candidates) is shipped. Be precise.

2. assistant (2026-06-13T19:55:37.524Z)

I'll map out the learned-LOVO surface for P3 fusion by reading the backend evaluation files you specified. Let me start by examining the core files.

3. user (2026-06-13T19:55:37.729Z)

```
1      """Distill Gemini (teacher) into a LEARNED local rally classifier (student).
2
3      Threshold-tuning the local knobs hit a low ceiling (see calibrate_local). This goes
4      further: learn "is this WASB candidate a real rally?" from Gemini silver labels,
5      using a few resolution/fps-INVARIANT local features, so the model transfers across
6      videos and runs 100% local at inference (no Gemini, privacy lever intact).
7
8      Flow (all local at runtime):
9          WASB trajectory -> recall-oriented candidate windows (propose many; classifier
10         filters) -> per-candidate features [duration, peak_velocity, mean_velocity,
11         active_ratio, net_crossings] -> LogisticRegression(prob >= threshold) -> merge
12         -> rally windows.
13
14         Training labels come from Gemini full-context labels via the SAME IoU matching the
15         evaluator uses (training.label_candidates). Honest leave-one-video-out: train on the
16         other videos' candidates, predict the held-out one, score vs its Gemini labels, and
17         compare against the threshold baseline. Reuses classifier.py, training.label_candidates,
18         rally_gate, metrics, calibrate_wasb.load_golden_gts, gemini_refine.merge_overlaps.
19
20         Run:
21             python -m backend.eval.distill_local --manifest videos.json --model-out
output/local_rally_model.json
22         """
```

```

23     from __future__ import annotations
24
25     import json
26     import argparse
27     from typing import Dict, List, Optional, Tuple
28
29
30     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
31     from backend.pipeline.detectors import rally_gate
32     from backend.eval.calibrate_wasb import load_golden_gts
33     from backend.eval.training import label_candidates
34     from backend.eval.classifier import LogisticRegression
35     from backend.eval.metrics import score
36     from backend.eval.gemini_refine import merge_overlaps
37
38     Interval = Tuple[float, float]
39
40     FEATURE_NAMES = ["duration", "peak_velocity", "mean_velocity", "active_ratio",
41 "net_crossings"]
42     # Recall-oriented proposal: deliberately permissive so real rallies are among the
43     # candidates (the classifier removes the junk). (velocity_thresh, merge_gap, min_dur)
44     PROPOSAL = (0.005, 1.0, 0.5)
45     BASELINE_LOCAL = (0.02, 2.0, 2.0) # today's production-ish windowing, no learning
46
47     def build_candidates(trajecory_csv: str, fps: float, frame_width: float,
48                         proposal: Tuple[float, float, float] = PROPOSAL) ->
49 Tuple[List[Interval], List[List[float]]]:
50     """WASB trajectory -> (candidate intervals, fps/resolution-invariant feature
51 rows)."""
52     points = TrackNetRunner.parse_trajectory_csv(trajecory_csv)
53     vt, mg, mwd = proposal
54     wins = TrackNetRunner.trajectory_to_action_windows(
55         points, fps=fps, frame_width=frame_width,
56         velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd)
57     intervals = [(w["start"], w["end"]) for w in wins]
58     gated = rally_gate.gate_windows(points, intervals, fps, min_crossings=0) # for net-
59 crossings
60     X: List[List[float]] = []
61     for w, g in zip(wins, gated):
62         dur = max(1e-6, w["end"] - w["start"])
63         win_frames = max(1.0, dur * fps)
64         X.append([
65             dur, # rally length (sec) ?
66             float(w["peak_velocity"]), # already normalized by
67             float(w["mean_velocity"]),
68             float(w["active_frame_count"]) / win_frames, # active-shuttle ratio ?
69             float(g.crossings), # net crossings (count) ?
70         ])
71     return intervals, X
72
73     def baseline_windows(trajecory_csv: str, fps: float, frame_width: float) ->
74 List[Interval]:
75     points = TrackNetRunner.parse_trajectory_csv(trajecory_csv)
76     vt, mg, mwd = BASELINE_LOCAL
77     wins = TrackNetRunner.trajectory_to_action_windows(
78         points, fps=fps, frame_width=frame_width,
79         velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd)
80     return [(w["start"], w["end"]) for w in wins]

```

```

79
80 def load_video(spec: Dict, iou: float = 0.5) -> Dict:
81     fps, fw = float(spec["fps"]), float(spec["frame_width"])
82     intervals, X = build_candidates(spec["trajectory"], fps, fw)
83     gts = load_golden_gts(spec["labels"])
84     y = label_candidates(intervals, gts, iou)
85     return {"name": spec.get("name", spec["trajectory"]), "intervals": intervals, "X":
X,
86         "y": y, "gts": gts, "baseline": baseline_windows(spec["trajectory"], fps,
fw)}
87
88
89 def predict_rallies(model: LogisticRegression, X: List[List[float]], intervals:
List[Interval],
90         threshold: float = 0.5) -> List[Interval]:
91     if not intervals:
92         return []
93     proba = model.predict_proba(X)
94     pos = [iv for iv, p in zip(intervals, proba) if p >= threshold]
95     return merge_overlaps(pos, gap_tol=1.0)
96
97
98 def run(specs: List[Dict], iou: float = 0.5, threshold: float = 0.5,
99         model_out: Optional[str] = None) -> dict:
100     videos = [load_video(s, iou) for s in specs]
101     print(f"=== Distilled local rally classifier vs Gemini silver-GT "
102           f"({len(videos)} videos, IoU>={iou}, prob>={threshold}) ===")
103     for v in videos:
104         ceil = score(v["intervals"], v["gts"], iou).recall
105         print(f"[{v['name']}] {len(v['intervals'])} candidates ({sum(v['y'])} pos) | "
106               f"proposal recall ceiling {ceil:.3f} | {len(v['gts'])} Gemini rallies")
107
108     result: dict = {"videos": [v["name"] for v in videos]}
109     if len(videos) >= 2:
110         print("\n--- HONEST leave-one-video-out: learned vs threshold baseline (held-
out) ---")
111         clf, base = [], []
112         for i, held in enumerate(videos):
113             others = [v for j, v in enumerate(videos) if j != i]
114             X = [row for v in others for row in v["X"]]
115             y = [t for v in others for t in v["y"]]
116             model = LogisticRegression().fit(X, y, FEATURE_NAMES)
117             preds = predict_rallies(model, held["X"], held["intervals"], threshold)
118             sc = score(preds, held["gts"], iou)
119             bsc = score(held["baseline"], held["gts"], iou)
120             clf.append(sc)
121             base.append(bsc)
122             print(f" held-out {held['name']}: LEARNED P={sc.precision:.3f}
R={sc.recall:.3f} F1={sc.f1:.3f}"
123                   f" | baseline P={bsc.precision:.3f} R={bsc.recall:.3f}
F1={bsc.f1:.3f}")
124         def mean(xs, a):
125             return sum(getattr(s, a) for s in xs) / len(xs)
126         print(f" MEAN held-out: LEARNED P={mean(clf, 'precision'):.3f}
R={mean(clf, 'recall'):.3f} "
127               f"F1={mean(clf, 'f1'):.3f} | baseline P={mean(base, 'precision'):.3f} "
128               f"R={mean(base, 'recall'):.3f} F1={mean(base, 'f1'):.3f}")
129         result["mean_learned_f1"] = mean(clf, "f1")
130         result["mean_baseline_f1"] = mean(base, "f1")
131
132     # Final model on ALL videos (for production / running on a fresh held-out video).
133     Xall = [row for v in videos for row in v["X"]]
134     yall = [t for v in videos for t in v["y"]]
135     final = LogisticRegression().fit(Xall, yall, FEATURE_NAMES)
136     assert final.w is not None # set by fit()

```

```

137     weights = dict(zip(FEATURE_NAMES, [round(float(w), 3) for w in final.w]))
138     print(f"\nfinal model standardized weights (sign/size = influence): {weights}")
139     if model_out:
140         final.save(model_out)
141         print(f"final model (all videos) -> {model_out}")
142         result["model_out"] = model_out
143     return result
144
145
146     def main() -> None:
147         ap = argparse.ArgumentParser(description="Distill Gemini labels into a local rally
classifier.")
148         ap.add_argument("--manifest", required=True,
149             help="JSON: [{name, trajectory, fps, frame_width, labels}, ...]
(labels = Gemini CSV)")
150         ap.add_argument("--iou", type=float, default=0.5)
151         ap.add_argument("--threshold", type=float, default=0.5, help="classifier probability
cutoff")
152         ap.add_argument("--model-out", default="output/local_rally_model.json")
153         args = ap.parse_args()
154         with open(args.manifest, encoding="utf-8") as f:
155             specs = json.load(f)
156         run(specs, iou=args.iou, threshold=args.threshold, model_out=args.model_out)
157
158
159     if __name__ == "__main__":
160         main()
161

```

4. user (2026-06-13T19:55:37.969Z)

```

1     """A tiny, dependency-free logistic-regression classifier (numpy only).
2
3     The offline learned segmenter (ADR-004) must run locally with no external API and
4     no heavy ML stack ? the project already depends on numpy, and sklearn is not
5     installed. This implements just enough: standardized features, balanced-class
6     gradient descent, sigmoid probabilities, and JSON (de)serialization so a trained
7     model is a small, human-readable, version-controllable file.
8
9     It deliberately stays simple: the rally-window features are few and roughly
10    linearly separable (high player motion + sustained activity => real rally), so a
11    well-regularized linear model is a sensible, inspectable baseline. Swap in a
12    richer model later behind the same fit/predict_proba/save/load interface.
13    """
14
15    import json
16    from typing import List, Optional
17
18    import numpy as np
19
20
21    class LogisticRegression:
22        def __init__(self, lr: float = 0.1, epochs: int = 1000, l2: float = 1e-3):
23            self.lr = lr
24            self.epochs = epochs
25            self.l2 = l2
26            self.w: Optional[np.ndarray] = None
27            self.b: float = 0.0
28            self.mean: Optional[np.ndarray] = None
29            self.std: Optional[np.ndarray] = None
30            self.feature_names: Optional[List[str]] = None
31
32        @staticmethod
33        def _sigmoid(z: np.ndarray) -> np.ndarray:
34            # Clip for numerical stability (avoid overflow in exp).

```

```

35         return 1.0 / (1.0 + np.exp(-np.clip(z, -30, 30)))
36
37     def _standardize(self, X: np.ndarray) -> np.ndarray:
38         return (X - self.mean) / self.std
39
40     def fit(self, X, y, feature_names: Optional[List[str]] = None,
41            class_weight: str = "balanced") -> "LogisticRegression":
42         """Fit with optional balanced class weighting (rally windows are imbalanced
43         ? many spurious candidates per real rally)."""
44         X = np.asarray(X, dtype=float)
45         y = np.asarray(y, dtype=float)
46         if X.ndim != 2 or X.shape[0] == 0:
47             raise ValueError("X must be a non-empty 2D array")
48         self.feature_names = feature_names
49
50         self.mean = X.mean(axis=0)
51         self.std = X.std(axis=0)
52         self.std[self.std == 0] = 1.0 # guard constant features
53         Xs = self._standardize(X)
54         n, d = Xs.shape
55
56         if class_weight == "balanced":
57             n_pos = max(1, int(y.sum()))
58             n_neg = max(1, int((1 - y).sum()))
59             w_pos, w_neg = n / (2.0 * n_pos), n / (2.0 * n_neg)
60         else:
61             w_pos = w_neg = 1.0
62         sw = np.where(y == 1, w_pos, w_neg)
63
64         self.w = np.zeros(d)
65         self.b = 0.0
66         for _ in range(self.epochs):
67             p = self._sigmoid(Xs @ self.w + self.b)
68             err = (p - y) * sw
69             grad_w = Xs.T @ err / n + self.l2 * self.w
70             grad_b = float(err.mean())
71             self.w -= self.lr * grad_w
72             self.b -= self.lr * grad_b
73         return self
74
75     def predict_proba(self, X) -> np.ndarray:
76         if self.w is None:
77             raise RuntimeError("model is not fitted")
78         Xs = self._standardize(np.asarray(X, dtype=float))
79         return self._sigmoid(Xs @ self.w + self.b)
80
81     def predict(self, X, threshold: float = 0.5) -> np.ndarray:
82         return (self.predict_proba(X) >= threshold).astype(int)
83
84     # --- persistence (small JSON, no pickle/joblib) -----
85
86     def to_dict(self) -> dict:
87         return {
88             "model": "logreg",
89             "lr": self.lr, "epochs": self.epochs, "l2": self.l2,
90             "w": self.w.tolist() if self.w is not None else None,
91             "b": self.b,
92             "mean": self.mean.tolist() if self.mean is not None else None,
93             "std": self.std.tolist() if self.std is not None else None,
94             "feature_names": self.feature_names,
95         }
96
97     @classmethod
98     def from_dict(cls, d: dict) -> "LogisticRegression":
99         m = cls(lr=d.get("lr", 0.1), epochs=d.get("epochs", 1000), l2=d.get("l2", 1e-3))

```

```

100         m.w = np.asarray(d["w"], dtype=float) if d.get("w") is not None else None
101         m.b = float(d.get("b", 0.0))
102         m.mean = np.asarray(d["mean"], dtype=float) if d.get("mean") is not None else
None
103         m.std = np.asarray(d["std"], dtype=float) if d.get("std") is not None else None
104         m.feature_names = d.get("feature_names")
105         return m
106
107     def save(self, path: str) -> None:
108         with open(path, "w") as f:
109             json.dump(self.to_dict(), f, indent=2)
110
111     @classmethod
112     def load(cls, path: str) -> "LogisticRegression":
113         with open(path) as f:
114             return cls.from_dict(json.load(f))
115

```

5. user (2026-06-13T19:55:38.174Z)

```

1     """Build a training set for the offline, API-free learned rally segmenter.
2
3     Phase 4 of the collector->indexer bridge (see docs/ROADMAP.md, ADR-004). The idea:
4     a detector (YOLO player-motion today, optionally WASB shuttle-motion later)
5     produces candidate rally windows with features; the collector's ground-truth
6     rally intervals tell us which windows are real. A lightweight classifier learns
7     that mapping, replacing the paid Gemini refinement with a local model.
8
9     This module is the *data layer* ? pure and unit-tested:
10
11     - `label_candidates` ? generic: label windows positive/negative by whether they
12       match a ground-truth rally (same IoU matching the evaluator uses, so training
13       labels are consistent with how we score).
14     - `extract_yolo_features` ? YOLO-specific feature vector from a candidate row's
15       persisted `stats`. Pluggable: a WASB substrate supplies its own feature_fn.
16     - `build_training_set` ? glue: rows + GT -> (X, y, intervals).
17
18     No model training or I/O here; that lives in the trainer/segmenter once the
19     substrate is chosen from the Phase 3 candidate-recall result.
20     """
21
22     from typing import Callable, Dict, List, Tuple
23
24     from backend.eval.metrics import Interval, match_intervals
25
26     # YOLO Stage-1 persists these per candidate window in `candidate_segments.stats`
27     # (see HybridYoloSegmenter). Duration is derived from the window bounds.
28     YOLO_FEATURE_NAMES = [
29         "duration", "peak_velocity", "mean_velocity", "active_frame_count",
"track_id_count",
30     ]
31
32
33     def extract_yolo_features(row: Dict) -> List[float]:
34         """Feature vector for one YOLO candidate row (dict from query_candidate_segments).
35
36         `stats` is already JSON-deserialized by the DB layer. Missing fields default
37         to 0.0 so a sparse/old row still yields a well-formed vector.
38         """
39         stats = row.get("stats") or {}
40         duration = float(row["end_time"]) - float(row["start_time"])
41         return [
42             duration,
43             float(stats.get("peak_velocity", 0.0)),
44             float(stats.get("mean_velocity", 0.0)),

```

```

45         float(stats.get("active_frame_count", 0)),
46         float(stats.get("track_id_count", 0)),
47     ]
48
49
50 def label_candidates(candidate_intervals: List[Interval], gt_intervals: List[Interval],
51                     iou_threshold: float = 0.5) -> List[int]:
52     """Label each candidate window 1 (real rally) or 0 (spurious).
53
54     A window is positive iff it is matched one-to-one to a ground-truth rally at
55     `iou_threshold` ? the *same* matching the evaluator uses for true positives,
56     so the classifier is trained on exactly what we score. Substrate-agnostic:
57     works for YOLO or WASB-derived windows alike.
58     """
59     mr = match_intervals(candidate_intervals, gt_intervals, iou_threshold)
60     positive = {m.pred_index for m in mr.matches}
61     return [1 if i in positive else 0 for i in range(len(candidate_intervals))]
62
63
64 def build_training_set(
65     candidate_rows: List[Dict],
66     gt_intervals: List[Interval],
67     iou_threshold: float = 0.5,
68     feature_fn: Callable[[Dict], List[float]] = extract_yolo_features,
69 ) -> Tuple[List[List[float]], List[int], List[Interval]]:
70     """Turn one video's candidate rows + ground truth into (X, y, intervals).
71
72     `feature_fn` is pluggable so a WASB substrate can supply shuttle-motion
73     features without changing the labeling logic. Returns the feature matrix `X`,
74     binary labels `y`, and the candidate intervals (handy for downstream merging).
75     """
76     intervals: List[Interval] = [
77         (float(r["start_time"]), float(r["end_time"])) for r in candidate_rows
78     ]
79     y = label_candidates(intervals, gt_intervals, iou_threshold)
80     X = [feature_fn(r) for r in candidate_rows]
81     return X, y, intervals
82

```

6. user (2026-06-13T19:55:38.339Z)

```

1     """Experiment harness ? A/B + shadow + side-by-side for rally detection.
2
3     This is the thin **variant/experiment layer** that `docs/EXPERIMENT_HARNESS.md`
4     identifies as the one missing piece: ~80% of the machinery (scoring, LOVO, the
5     no-regression gate, the pinnable classifier artifact, the persisted-candidate
6     substrate) already exists ? this module composes it. **No new scoring math lives
7     here**; everything defers to:
8
9     - `backend.eval.metrics`      ? `score`, `match_intervals`, `interval_iou`
10    - `backend.eval.training`      ? `label_candidates` (y by the same IoU matcher)
11    - `backend.eval.classifier`    ? `LogisticRegression` (the pinnable JSON model)
12    - `backend.eval.gemini_refine` ? `merge_overlaps` (the blessed window merge)
13    - `backend.eval.regression`    ? `gt_hash` (label-set fingerprint)
14    - `backend.eval.calibration`   ? `pct_improvement`
15
16    The thesis (`EXPERIMENT_HARNESS.md` §2): separate the **expensive, risky DETECTION**
17    (per-frame signals ? run once on the GPU/WSL box, persisted into
18    `candidate_segments.stats`) from the **cheap, swappable DECISION** (given those
19    signals, where are the rallies). A `Variant` is a declarative re-scoring recipe;
20    given persisted candidate features it produces `List[Interval]` deterministically,
21    with no GPU and zero production risk. So most experiments are pure offline
22    re-scoring of stored data and never touch the served pipeline.
23
24    Three modes (`EXPERIMENT_HARNESS.md` §5):

```

```

25     - `compare()` ? labeled A/B vs golden labels: per-video + LOVO-honest
26     aggregate deltas. Mirrors `distill_local`'s honest train-on-others/predict-
27     held-out loop, but variant-parameterised.
28     - `compare_unlabeled()` ? SxS on NEW footage with no ground truth: where do two
29     variants agree / diverge (A-only, B-only, agreed). The divergences are what a
30     human spot-checks (and what two highlight reels would show side by side).
31     - the promotion gate stays `run_regression.py` + `regression_baseline.json`
32     (exit 0/1/3); **rollback = flip the variant/config, never `git revert`.**
33
34 Pure + offline + unit-tested. The only DB-touching helper is
35 `load_video_candidates`, which reads persisted candidates via
36 `Database.query_candidate_segments`.
37 """
38 from __future__ import annotations
39
40 import json
41 import logging
42 import os
43 from dataclasses import dataclass, field
44 from datetime import datetime, timezone
45 from hashlib import sha1
46 from typing import Any, Dict, List, Optional, Sequence
47
48 from backend.eval.calibration import pct_improvement
49 from backend.eval.classifier import LogisticRegression
50 from backend.eval.gemini_refine import merge_overlaps
51 from backend.eval.metrics import Interval, match_intervals, score
52 from backend.eval.regression import gt_hash
53 from backend.eval.training import label_candidates
54
55 logger = logging.getLogger(__name__)
56
57 # Where a comparison run appends one reproducible record. Tracked location (NOT under
58 # the gitignored output/) so the leaderboard survives a clean checkout, mirroring
59 # regression.DEFAULT_BASELINE_PATH.
60 DEFAULT_LEADERBOARD_PATH = os.path.join("eval_baselines", "experiment_leaderboard.json")
61 _METRICS = ("precision", "recall", "f1", "mean_iou")
62
63
64 class ExperimentError(ValueError):
65     """A variant cannot be evaluated as configured (e.g. a learned variant asked to
66     score an unlabeled video with no pinned model, or a gate referencing an absent
67     feature)."""
68
69
70 # ----- Variant
71
72
73 @dataclass
74 class Variant:
75     """A declarative re-scoring recipe ? NOT a code branch (`EXPERIMENT_HARNESS.md` §4).
76
77     A variant turns one video's persisted candidate windows + features into rally
78     intervals. Every knob defaults to the **control** behaviour (no gate, no learned
79     filter), so a variant that merely *carries* a new, disabled cue is byte-identical
80     to control ? the core no-regression guarantee.
81
82     Fields:
83     - ``segmenter`` / ``config_overrides`` / ``cue_flags`` ? informational provenance
84       for the leaderboard and for selecting the served path (the existing
85       `segmenter_registry` + `IndexingConfig` do the actual selection in production).
86       New cues are recorded here default-OFF.
87     - ``min_crossings`` ? decision-gate **Gate A**: keep only windows whose persisted
88       ``net_crossings`` feature is >= this. 0 = off. This is the eval-only gate from
89       `rally_gate.py` expressed as a re-scoring knob (kills service-feed / held-

```



```

shuttle
90         windows ? see `MULTI_SIGNAL_FUSION_PLAN.md` §3.1).
91     - ``min_players`` ? decision-gate **Gate B** (the future person cue): keep windows
92       whose persisted ``players_in_court`` is >= this. 0 = off (no-op until the person
93       cue persists that feature).
94     - ``classifier_model`` ? path to a frozen `LogisticRegression` JSON. When set,
95       `compare()` scores videos with the SHIPPED model as-is (no per-fold training);
96       required for `compare_unlabeled` on a learned variant.
97     - ``feature_names`` ? the persisted features the learned filter consumes. When set
98       WITHOUT ``classifier_model``, `compare()` trains a fresh model per LOVO fold
99       (honest) ? the recipe, not a pinned artifact.
100    - ``threshold`` ? classifier probability cutoff. ``merge_gap`` ? post-filter
101      overlap-merge gap (seconds), the same `gemini_refine.merge_overlaps` default.
102    """
103
104    name: str
105    segmenter: str = "wasb_hybrid"
106    config_overrides: Dict[str, Any] = field(default_factory=dict)
107    cue_flags: Dict[str, bool] = field(default_factory=dict)
108    min_crossings: int = 0
109    min_players: int = 0
110    classifier_model: Optional[str] = None
111    feature_names: Optional[List[str]] = None
112    threshold: float = 0.5
113    merge_gap: float = 1.0
114
115    def is_learned(self) -> bool:
116        """True if this variant applies a learned classifier (pinned model or
117        recipe)."""
118        return self.classifier_model is not None or bool(self.feature_names)
119
120    def to_dict(self) -> dict:
121        return {
122            "name": self.name,
123            "segmenter": self.segmenter,
124            "config_overrides": self.config_overrides,
125            "cue_flags": self.cue_flags,
126            "min_crossings": self.min_crossings,
127            "min_players": self.min_players,
128            "classifier_model": self.classifier_model,
129            "feature_names": self.feature_names,
130            "threshold": self.threshold,
131            "merge_gap": self.merge_gap,
132        }
133
134    @classmethod
135    def from_dict(cls, d: dict) -> "Variant":
136        known = {f for f in cls.__dataclass_fields__} # type: ignore[attr-defined]
137        return cls(**{k: v for k, v in d.items() if k in known})
138
139    def spec_hash(self) -> str:
140        """Stable 12-char hash of the recipe ? identifies the variant in the leaderboard
141        and makes a result reproducible. The pinned **control** variant always hashes
142        the
143        same, so a regression is always traceable to a recipe change, never to drift."""
144        blob = json.dumps(self.to_dict(), sort_keys=True, default=str)
145        return sha1(blob.encode()).hexdigest()[:12]
146
147    #: The pinned, frozen baseline: candidates as detected, merged, no gate, no learned
148    #: filter. Always the comparison reference; "rollback" = make this the served variant.
149    CONTROL = Variant(name="control")
150
151    # ----- persisted candidates

```

```

152
153
154 @dataclass
155 class VideoCandidates:
156     """One video's persisted detection, ready for offline re-scoring.
157
158     ``intervals`` are the candidate windows; ``features`` is column-major
159     (``feature_name`` -> per-candidate value, each list aligned to ``intervals``);
160     ``gts`` are golden labels (None for new/unlabeled footage). Build one from the DB
161     with ``load_video_candidates``, or directly in tests.
162     """
163
164     name: str
165     intervals: List[Interval]
166     features: Dict[str, List[float]] = field(default_factory=dict)
167     gts: Optional[List[Interval]] = None
168
169
170 def _feature_column(vc: VideoCandidates, name: str) -> List[float]:
171     """Persisted feature column, defaulting missing/short columns to 0.0 (matches
172     ``training.extract_yolo_features``, which lets a sparse/old row still vectorise)."""
173     col = vc.features.get(name)
174     n = len(vc.intervals)
175     if col is None:
176         logger.warning("variant references feature %r absent from %s ? treating as 0.0",
177                        name, vc.name)
178         return [0.0] * n
179     if len(col) != n:
180         raise ExperimentError(
181             f"feature {name!r} has {len(col)} values but {vc.name} has {n} candidates")
182     return [float(x) for x in col]
183
184
185 def _feature_matrix(vc: VideoCandidates, feature_names: Sequence[str]) ->
List[List[float]]:
186     cols = [_feature_column(vc, name) for name in feature_names]
187     return [list(row) for row in zip(*cols)] if cols else [[] for _ in vc.intervals]
188
189
190 def _gate_mask(variant: Variant, vc: VideoCandidates) -> List[bool]:
191     """Boolean keep-mask from the decision gates (Gate A net-crossings, Gate B
192     players)."""
193     n = len(vc.intervals)
194     mask = [True] * n
195     if variant.min_crossings > 0:
196         col = _feature_column(vc, "net_crossings")
197         mask = [m and (col[i] >= variant.min_crossings) for i, m in enumerate(mask)]
198     if variant.min_players > 0:
199         col = _feature_column(vc, "players_in_court")
200         mask = [m and (col[i] >= variant.min_players) for i, m in enumerate(mask)]
201     return mask
202
203
204 def predict(variant: Variant, vc: VideoCandidates,
205             model: Optional[LogisticRegression] = None) -> List[Interval]:
206     """Variant prediction: gate by persisted features, optionally filter by a learned
207     model, then merge. Pure and deterministic.
208
209     ``model`` (an already-fitted ``LogisticRegression``) is supplied by ``compare`` per LOVO
210     fold; if omitted and the variant pins ``classifier_model``, that frozen model is
211     loaded. A learned variant with neither a supplied nor a pinned model raises ? there
212     is nothing to score with.
213     """
214     mask = _gate_mask(variant, vc)
215     if variant.feature_names:

```

```

215         if model is None:
216             if variant.classifier_model is None:
217                 raise ExperimentError(
218                     f"variant {variant.name!r} is learned but has no model to predict "
219                     f"with (pass a fitted model or set classifier_model)")
220                 model = LogisticRegression.load(variant.classifier_model)
221             proba = model.predict_proba(_feature_matrix(vc, variant.feature_names))
222             mask = [m and (float(proba[i]) >= variant.threshold) for i, m in
enumerate(mask)]
223         kept = [iv for iv, keep in zip(vc.intervals, mask) if keep]
224         return merge_overlaps(kept, gap_tol=variant.merge_gap)
225
226
227     # ----- variant scoring
228
229
230     def _train(variant: Variant, videos: Sequence[VideoCandidates], iou: float) ->
LogisticRegression:
231         """Fit the variant's classifier on the pooled candidates of ``videos`` (labels via
the
232         evaluator's own IoU matcher). The honest-LOVO training step from `distill_local`. """
233         X: List[List[float]] = []
234         y: List[int] = []
235         for vc in videos:
236             if vc.gts is None:
237                 raise ExperimentError(f"cannot train: {vc.name} has no golden labels")
238             X.extend(_feature_matrix(vc, variant.feature_names or []))
239             y.extend(label_candidates(vc.intervals, vc.gts, iou))
240         if not X:
241             raise ExperimentError("cannot train a learned variant on zero candidates")
242         return LogisticRegression().fit(X, y, variant.feature_names)
243
244
245     def evaluate_variant(videos: Sequence[VideoCandidates], variant: Variant,
246                         iou: float = 0.5, honest: bool = True) -> Dict[str, Any]:
247         """Score a variant on a labeled corpus. Returns per-video Scores + predictions + a
248         mean aggregate.
249
250         Prediction policy:
251         - **static variant** (no learned filter): predict each video directly ? no
training,
252         so no leakage; per-video scoring is already honest.
253         - **learned, pinned model** (``classifier_model`` set): score every video with the
SHIPPED model. ? optimistic if those videos were in its training set ? use this
254         to
255         report "how the shipped artifact does here", not for the promotion decision.
256         - **learned recipe** (``feature_names``, no pinned model) + ``honest=True``: LOVO
?
257         train on the OTHER videos, predict the held-out one. The number to trust.
258         - learned recipe + ``honest=False``: train on all, predict each (overfit; sanity
only).
259         """
260         for vc in videos:
261             if vc.gts is None:
262                 raise ExperimentError(f"{vc.name} has no golden labels; use
compare_unlabeled")
263
264         per_video: List[Dict[str, Any]] = []
265         predictions: Dict[str, List[Interval]] = {}
266
267         recipe_lovo = bool(variant.feature_names) and variant.classifier_model is None
268         pinned_model = (LogisticRegression.load(variant.classifier_model)
269                         if variant.classifier_model is not None else None)
270         all_model = None
271         if recipe_lovo and not honest:

```

```

272         all_model = _train(variant, videos, iou)
273
274     for i, vc in enumerate(videos):
275         if recipe_lovo and honest:
276             others = [v for j, v in enumerate(videos) if j != i]
277             if not others:
278                 raise ExperimentError("LOVO needs >=2 videos; pass honest=False or a
pinned model")
279             model: Optional[LogisticRegression] = _train(variant, others, iou)
280         elif recipe_lovo:
281             # honest=False ? one model over all
282             model = all_model
283         else:
284             # static variant or pinned model
285             model = pinned_model
286         preds = predict(variant, vc, model=model)
287         assert vc.gts is not None
288         # guarded at function top
289         sc = score(preds, vc.gts, iou)
290         per_video.append({"video": vc.name, "num_pred": len(preds), **{m: getattr(sc, m)
for m in _METRICS}})
291         predictions[vc.name] = preds
292
293     aggregate = {m: (sum(p[m] for p in per_video) / len(per_video) if per_video else
0.0)
for m in _METRICS}
294
295     return {
296         "variant": variant.name,
297         "spec_hash": variant.spec_hash(),
298         "is_learned": variant.is_learned(),
299         "honest_lovo": recipe_lovo and honest,
300         "per_video": per_video,
301         "aggregate": aggregate,
302         "predictions": predictions,
303     }
304
305 def compare(videos: Sequence[VideoCandidates], variant_a: Variant, variant_b: Variant,
306            iou: float = 0.5, honest: bool = True) -> Dict[str, Any]:
307     """Offline labeled A/B (`EXPERIMENT_HARNESS.md` §5.1). Scores both variants on the
same
308     golden corpus and reports the **B ? A** deltas, per video and in aggregate.
309
310     The deltas use the existing `metrics.score` (per video) +
`calibration.pct_improvement`;
311     learned variants are scored LOVO-honestly. The result is a self-contained record fit
for
312     `record_experiment` ? variant specs + hashes + the label-set fingerprint travel with
it.
313
314     """
315     if len(videos) < 1:
316         raise ExperimentError("compare needs at least one labeled video")
317     eval_a = evaluate_variant(videos, variant_a, iou, honest)
318     eval_b = evaluate_variant(videos, variant_b, iou, honest)
319
320     delta = {m: eval_b["aggregate"][m] - eval_a["aggregate"][m] for m in _METRICS}
321     delta_pct = {m: pct_improvement(eval_a["aggregate"][m], eval_b["aggregate"][m]) for
m in _METRICS}
322
323     by_video_a = {p["video"]: p for p in eval_a["per_video"]}
324     per_video_delta = [
325         {"video": p["video"], **{m: p[m] - by_video_a[p["video"]][m] for m in _METRICS}}
326         for p in eval_b["per_video"]
327     ]
328
329     # One fingerprint over the whole label set ? detects "the deltas were measured
against

```

```

327     # different labels" the same way regression.gt_hash guards the no-regression
baseline.
328     all_gts: List[Interval] = [iv for vc in videos for iv in (vc.gts or [])]
329
330     return {
331         "iou": iou,
332         "label_set_hash": gt_hash(all_gts),
333         "num_videos": len(videos),
334         "variant_a": eval_a,
335         "variant_b": eval_b,
336         "delta": delta,
337         "delta_pct": delta_pct,
338         "per_video_delta": per_video_delta,
339     }
340
341
342     # ----- SxS on unlabeled video
343
344
345     def compare_unlabeled(video: VideoCandidates, variant_a: Variant, variant_b: Variant,
346                          agree_iou: float = 0.5) -> Dict[str, Any]:
347         """Side-by-side on NEW footage with no ground truth (`EXPERIMENT_HARNESS.md` §5.2).
348
349         With no labels there is no lift to measure ? instead this surfaces where the two
350         variants **agree vs diverge**, which is exactly what a human reviews (and what two
351         highlight reels would show side by side):
352
353         - ``agreed`` ? windows both variants found (matched at ``agree_iou``);
354         - ``a_only`` ? A fired, B suppressed (B's added precision, if A was over-firing);
355         - ``b_only`` ? B fired, A did not (B's new detections ? real recall or new FPS:
356           the human / a later golden label adjudicates).
357
358         Both variants must be predictable without labels: a learned variant therefore needs
359         a
360         pinned ``classifier_model`` (you cannot train on an unlabeled video). Reuses
361         ``metrics.match_intervals`` ? no new matching logic.
362
363         """
364         preds_a = predict(variant_a, video)
365         preds_b = predict(variant_b, video)
366         mr = match_intervals(preds_a, preds_b, agree_iou)
367
368         agreed = [{"a": preds_a[m.pred_index], "b": preds_b[m.gt_index], "iou": m.iou}
369                  for m in mr.matches]
370         agree_iou = [m.iou for m in mr.matches]
371         a_only = [preds_a[i] for i in mr.unmatched_pred_indices]
372         b_only = [preds_b[i] for i in mr.unmatched_gt_indices]
373
374         def _dur(ivs: List[Interval]) -> float:
375             return sum(e - s for s, e in ivs)
376
377         return {
378             "video": video.name,
379             "agree_iou": agree_iou,
380             "variant_a": variant_a.name,
381             "variant_b": variant_b.name,
382             "num_a": len(preds_a),
383             "num_b": len(preds_b),
384             "num_agreed": len(agreed),
385             "mean_agree_iou": (sum(agree_iou) / len(agree_iou)) if agree_iou else 0.0,
386             "duration_a": _dur(preds_a),
387             "duration_b": _dur(preds_b),
388             "agreed": agreed,
389             "a_only": a_only,
390             "b_only": b_only,
391         }

```

```

390
391
392 # ----- leaderboard / DB
393
394
395 def record_experiment(result: Dict[str, Any], path: str = DEFAULT_LEADERBOARD_PATH,
396                     timestamp: Optional[str] = None) -> Dict[str, Any]:
397     """Append a compact, reproducible record of a `compare()` result to the JSON
398     leaderboard (`EXPERIMENT_HARNESS.md` §6: experiments are records). Generalises
399     `regression_baseline.json`; deliberately the size of a baseline file, not a service
400     (§9). Returns the row written. ``timestamp`` is injectable for deterministic tests.
401     """
402     row: Dict[str, Any] = {
403         "timestamp": timestamp or datetime.now(timezone.utc).isoformat(),
404         "iou": result.get("iou"),
405         "label_set_hash": result.get("label_set_hash"),
406         "num_videos": result.get("num_videos"),
407         "variant_a": {"name": result["variant_a"]["variant"],
408                     "spec_hash": result["variant_a"]["spec_hash"],
409                     "aggregate": result["variant_a"]["aggregate"]},
410         "variant_b": {"name": result["variant_b"]["variant"],
411                     "spec_hash": result["variant_b"]["spec_hash"],
412                     "aggregate": result["variant_b"]["aggregate"]},
413         "delta": result.get("delta"),
414     }
415     entries: List[Dict[str, Any]] = []
416     if os.path.isfile(path):
417         with open(path, encoding="utf-8") as f:
418             entries = json.load(f)
419     entries.append(row)
420     os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
421     with open(path, "w", encoding="utf-8") as f:
422         json.dump(entries, f, indent=2)
423     logger.info("recorded experiment %s vs %s (?f1=%3f) -> %s",
424               row["variant_a"]["name"], row["variant_b"]["name"],
425               (result.get("delta") or {}).get("f1", 0.0), path)
426     return row
427
428
429 def load_video_candidates(db: Any, video_id: str, *, detector: Optional[str] = None,
430                         gts: Optional[List[Interval]] = None,
431                         feature_names: Sequence[str] = ()) -> VideoCandidates:
432     """Bridge persisted detection ? `VideoCandidates` via
433     `Database.query_candidate_segments`.
434     ``feature_names`` selects which persisted features to lift out of each row's
435     ``stats``
436     JSON (``"duration"`` is derived from the window bounds). This is the only DB-
437     touching
438     function; the comparison itself is pure and offline. Exercised owner-side on a real
439     DB,
440     but works against any `Database` (including an in-memory one in tests).
441     """
442     rows = db.query_candidate_segments(video_id, detector=detector)
443     intervals: List[Interval] = [(float(r["start_time"]), float(r["end_time"])) for r in
444     rows]
445     features: Dict[str, List[float]] = {}
446     for name in feature_names:
447         if name == "duration":
448             features[name] = [float(r["end_time"]) - float(r["start_time"]) for r in
449             rows]
450         else:
451             features[name] = [float((r.get("stats") or {}).get(name, 0.0)) for r in
452             rows]
453     return VideoCandidates(name=video_id, intervals=intervals, features=features,

```

```
gts=gts)
448
```

7. assistant (2026-06-13T19:55:40.168Z)

Now let me search for files related to fusion features and person detection to understand the feature landscape:

8. user (2026-06-13T19:55:40.505Z)

```
backend\pipeline\detectors\fusion_features.py
backend\pipeline\segmenters\fusion_hybrid.py
```

9. user (2026-06-13T19:55:40.521Z)

No files found

10. user (2026-06-13T19:55:40.644Z)

No files found

11. assistant (2026-06-13T19:55:41.713Z)

Let me read the fusion_features.py and fusion_hybrid.py:

12. user (2026-06-13T19:55:42.133Z)

```
1      """Pure per-window person-cue features for multi-signal rally fusion
(MULTI_SIGNAL_FUSION_PLAN §3.2/§4).
2
3      NO I/O, no torch, no cv2 ? mirrors `rally_gate.py`'s discipline so the whole feature
4      math is unit-testable in a GPU-free dev container. The detection (person boxes,
5      shuttle trajectory) is produced elsewhere (`PersonDetector`, the WASB runner); this
6      module only turns those signals into a handful of scale/translation-invariant
7      features (counts / ratios / zone-membership ? never raw pixel coordinates, which would
8      memorise this court/camera and fail to generalise off a small labelled set).
9
10     Conventions:
11     - A person box is `(x1, y1, x2, y2)` in pixels; `(track_id, box)` when motion
needs it.
12     - per_frame is `{frame_idx: [(track_id, box), ...]}` keyed by the ORIGINAL video
frame
13     index (so it aligns directly with the shuttle trajectory's .frame ? no clip re-
timing).
14     - A shuttle point has .frame, .visible, .x, .y (pixels) ? the WASB
trajectory shape.
15     - court_polygon is normalised 0-1 [(x, y), ...] or None (None ? no mask,
every
16     detection counts ? the player-count-only mode).
17     - net is (axis 'x'|'y', position 0-1 normalised) or None (None ? two-sided =
0.0).
18
19     Every function degrades gracefully (returns 0.0 / empty) on missing inputs; outputs are
20     floats in [0, 1] except mean_player_motion (a non-negative normalised speed).
21     """
22     from __future__ import annotations
23
24     from typing import Dict, List, Optional, Sequence, Tuple
25
26     BBox = Tuple[float, float, float, float]
27     Net = Tuple[str, float]
28
29
30     def point_in_polygon(point: Tuple[float, float], polygon: Sequence[Tuple[float, float]])
```

```

-> bool:
31     """Ray-casting point-in-polygon. `point` and `polygon` share a coordinate space
32     (we use normalised 0-1). Empty/degenerate polygon (<3 vertices) ? False."""
33     if polygon is None or len(polygon) < 3:
34         return False
35     x, y = point
36     inside = False
37     n = len(polygon)
38     j = n - 1
39     for i in range(n):
40         xi, yi = polygon[i]
41         xj, yj = polygon[j]
42         # Does the horizontal ray at y cross edge (i, j)?
43         if (yi > y) != (yj > y):
44             x_cross = (xj - xi) * (y - yi) / (yj - yi) + xi if yj != yi else xi
45             if x < x_cross:
46                 inside = not inside
47         j = i
48     return inside
49
50
51     def _foot_norm(box: BBox, frame_width: float, frame_height: float) -> Tuple[float,
float]:
52         """Normalised foot point (bottom-centre) of a person box ? where the player stands,
53         the right anchor for court membership. Returns (0,0) if frame dims are unusable."""
54         if frame_width <= 0 or frame_height <= 0:
55             return (0.0, 0.0)
56         x1, _y1, x2, y2 = box
57         return ((x1 + x2) / 2.0 / frame_width, y2 / frame_height)
58
59
60     def _center_norm(box: BBox, frame_width: float, frame_height: float) -> Tuple[float,
float]:
61         if frame_width <= 0 or frame_height <= 0:
62             return (0.0, 0.0)
63         x1, y1, x2, y2 = box
64         return ((x1 + x2) / 2.0 / frame_width, (y1 + y2) / 2.0 / frame_height)
65
66
67     def person_in_court(box: BBox, frame_width: float, frame_height: float,
68                        court_polygon: Optional[Sequence[Tuple[float, float]]]) -> bool:
69         """Is this person on the court? Foot-point-in-polygon when a polygon is supplied;
70         True (count everyone) when it is None. Unusable frame dims (w/h <= 0) make the foot
71         point meaningless, so a masked check returns False rather than testing a bogus
72         (0,0)."""
73         if court_polygon is None:
74             return True
75         if frame_width <= 0 or frame_height <= 0:
76             return False
77         return point_in_polygon(_foot_norm(box, frame_width, frame_height), court_polygon)
78
79     def in_court_counts(per_frame: Dict[int, List[Tuple[int, BBox]]], frame_width: float,
80                       frame_height: float,
81                       court_polygon: Optional[Sequence[Tuple[float, float]]]) ->
List[int]:
82         """Per-frame count of in-court persons (one entry per sampled frame)."""
83         return [
84             sum(1 for _tid, box in dets if person_in_court(box, frame_width, frame_height,
85             court_polygon))
86             for _frame_idx, dets in sorted(per_frame.items())
87         ]
88
89     def _median(vals: Sequence[float]) -> float:

```



```

90         if not vals:
91             return 0.0
92         s = sorted(vals)
93         n = len(s)
94         return float(s[n // 2] if n % 2 else 0.5 * (s[n // 2 - 1] + s[n // 2]))
95
96
97     def players_in_court(counts: Sequence[int]) -> float:
98         """Median per-frame in-court count (?2 singles / 4 doubles; 0-1 = dead time)."""
99         return _median(list(counts))
100
101
102     def in_court_ratio(counts: Sequence[int], min_players: int = 2) -> float:
103         """Fraction of sampled frames with >= `min_players` in court (track stability vs
104 flicker)."""
105         if not counts:
106             return 0.0
107         return sum(1 for c in counts if c >= min_players) / len(counts)
108
109     def mean_player_motion(per_frame: Dict[int, List[Tuple[int, BBox]]],
110                             frame_width: float, frame_height: float) -> float:
111         """Mean per-track centre displacement between consecutive sampled frames, with x
112 normalised by frame width and y by frame height (per-axis; uniform-scale-invariant,
113 not aspect-ratio-isotropic). 0.0 if <2 frames or no shared tracks."""
114         if frame_width <= 0 or len(per_frame) < 2:
115             return 0.0
116         frames = sorted(per_frame.keys())
117         steps: List[float] = []
118         for a, b in zip(frames, frames[1:]):
119             prev = {tid: box for tid, box in per_frame[a]}
120             for tid, box in per_frame[b]:
121                 if tid in prev:
122                     cx0, cy0 = _center_norm(prev[tid], frame_width, frame_height)
123                     cx1, cy1 = _center_norm(box, frame_width, frame_height)
124                     steps.append(((cx1 - cx0) ** 2 + (cy1 - cy0) ** 2) ** 0.5)
125         if not steps:
126             return 0.0
127         return sum(steps) / len(steps)
128
129
130     def two_sided_motion(per_frame: Dict[int, List[Tuple[int, BBox]]], frame_width: float,
131                             frame_height: float, net: Optional[Net],
132                             court_polygon: Optional[Sequence[Tuple[float, float]]]) -> float:
133         """Fraction of sampled frames with >=1 in-court person on BOTH net halves.
134
135 A real rally is two-sided; a single player feeding/holding the shuttle is one-sided.
136 The net splits the play axis at `net=(axis, position)` in normalised coords. None ?
137 0.0.
138 """
139         if net is None or not per_frame:
140             return 0.0
141         axis, pos = net
142         both = 0
143         for _frame_idx, dets in per_frame.items():
144             near, far = False, False
145             for _tid, box in dets:
146                 if not person_in_court(box, frame_width, frame_height, court_polygon):
147                     continue
148                 fx, fy = _foot_norm(box, frame_width, frame_height)
149                 coord = fx if axis == "x" else fy
150                 if coord < pos:
151                     near = True
152                 else:
153                     far = True

```

```

153         if near and far:
154             both += 1
155     return both / len(per_frame)
156
157
158     def _shuttle_in_box(sx: float, sy: float, box: BBox, frame_width: float, frame_height:
float,
159                        pad_frac: float) -> bool:
160         """Is the (pixel) shuttle point inside a person box, expanded by `pad_frac` of frame
width/height (the 'adjacent / in-hand' tolerance)?"""
161         x1, y1, x2, y2 = box
162         px = pad_frac * frame_width
163         py = pad_frac * frame_height
164         return (x1 - px) <= sx <= (x2 + px) and (y1 - py) <= sy <= (y2 + py)
165
166
167     def shuttle_player_colocation(per_frame: Dict[int, List[Tuple[int, BBox]]],
shuttle_points,
168                                  frame_width: float, frame_height: float,
169                                  pad_frac: float = 0.03) -> float:
170         """Fraction of (visible shuttle frames that were also person-sampled) in which the
shuttle sits inside/adjacent to a person box. **held/in-hand ? high; in-flight ?
~0**
171         ? the direct held-shuttle discriminator. 0.0 if no aligned visible shuttle frames.
172         """
173         if not per_frame or frame_width <= 0 or frame_height <= 0:
174             return 0.0
175         aligned = 0
176         collocated = 0
177         for p in shuttle_points or []:
178             if not getattr(p, "visible", False):
179                 continue
180             dets = per_frame.get(int(p.frame))
181             if dets is None:
182                 # shuttle frame wasn't person-sampled ? can't judge
183                 continue
184             aligned += 1
185             if any(_shuttle_in_box(p.x, p.y, box, frame_width, frame_height, pad_frac)
186                   for _tid, box in dets):
187                 collocated += 1
188         return collocated / aligned if aligned else 0.0
189
190
191     # The ~5 person features persisted into candidate `stats` (order = the learned-fusion
192     # feature columns appended after the shuttle features in distill_local.FEATURE_NAMES).
193     PERSON_FEATURE_NAMES = [
194         "players_in_court", "two_sided_motion", "mean_player_motion",
195         "in_court_ratio", "shuttle_player_colocation",
196     ]
197
198
199     def compute_person_features(per_frame: Dict[int, List[Tuple[int, BBox]]],
shuttle_points,
200                                  frame_width: float, frame_height: float,
201                                  court_polygon: Optional[Sequence[Tuple[float, float]]] =
None,
202                                  net: Optional[Net] = None,
203                                  min_players: int = 2) -> Dict[str, float]:
204         """Aggregate the per-window person features from per-frame detections + the in-
window
205         shuttle trajectory. Pure; returns exactly `PERSON_FEATURE_NAMES`, all floats."""
206         counts = in_court_counts(per_frame, frame_width, frame_height, court_polygon)
207         return {
208             "players_in_court": players_in_court(counts),
209             "two_sided_motion": two_sided_motion(per_frame, frame_width, frame_height, net,
court_polygon),

```

```

211         "mean_player_motion": mean_player_motion(per_frame, frame_width, frame_height),
212         "in_court_ratio": in_court_ratio(counts, min_players),
213         "shuttle_player_colocation": shuttle_player_colocation(
214             per_frame, shuttle_points, frame_width, frame_height),
215     }
216

```

13. user (2026-06-13T19:55:42.367Z)

```

1     """Multi-signal fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2).
2
3     Shuttle stays PRIMARY: this reuses an existing trajectory substrate (WASB by default,
4     TrackNet optional) to seed candidate windows exactly as `wasb_hybrid`/`tracknet_hybrid`
5     do ? it subclasses the same `TrajectoryHybridSegmenter` engine. It then ENRICHES each
6     shuttle-seeded window via two overridable seams the engine exposes:
7
8     - ``_enrich_candidate_stats`` ? always adds the net-crossing count (Gate A signal,
9     GPU-free, computed from the trajectory we already have); and, only when the person cue
10    is explicitly enabled (``indexing.fusion.cue_flags.person``), the person-cue features
11    (``fusion_features``), persisting them into ``candidate_segments.stats`` for the learned
12    fusion (P3). The person path lazily builds ONE `PersonDetector` and only then loads
the
13    torchvision/supervision model ? so the default (person-OFF) path never loads a
detector
14    model nor touches the GPU. (torch/cv2 themselves are imported by the registry
regardless,
15    via the pre-existing yolo_hybrid ? person_detector chain; the lazy part is the model.)
16    - ``_select_for_handoff`` ? optional served Gate A/B: when
``indexing.fusion.min_crossings``
17    (and, with the person cue, ``min_players``) are raised above 0, sub-threshold windows
are
18    dropped from the AI handoff (the held-shuttle / one-sided-feed false-positive fix).
19    **Persisted candidates remain the full superset** regardless, so the offline
experiment
20    harness can re-score any variant.
21
22    Defaults reproduce the substrate exactly: net_crossings/person features are persisted
but
23    nothing is gated (thresholds 0, person OFF), so `FusionSegmenter` with default config
seeds
24    and hands off identically to `wasb_hybrid`. Registered as ``fusion_hybrid``, NOT the
default
25    segmenter ? it is opt-in. Promotion to default happens only on measured LOVO lift
through the
26    experiment harness (EXPERIMENT_HARNESS.md), never silently.
27    """
28    from typing import Any, Dict, List, Optional, Tuple
29
30    from backend.pipeline.segmenters.base import segmenter_registry
31    from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
32    from backend.pipeline.detectors.base import DetectorRunner
33    from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
34    from backend.pipeline.detectors.tracknet_runner import TrackNetRunner, TrackNetConfig
35    from backend.pipeline.detectors import rally_gate
36    from backend.pipeline.detectors import fusion_features as ff
37
38
39    class FusionSegmenter(TrajectoryHybridSegmenter):
40        """Shuttle trajectory (primary) + net-crossing Gate A + optional person cue (Gate
B)."""
41
42        family = "fusion"
43        display_label = "Fusion"
44
45        def __init__(self, db: Any, config: Any):

```

```

46         super().__init__(db, config)
47         # Built lazily only if the person cue is enabled (no detector model loaded
otherwise).
48         self._person: Optional[Any] = None
49         # Cache the play-axis/net estimate per trajectory so rally_gate doesn't re-
estimate it
50         # over the whole trajectory once per candidate window (it depends only on
`points`).
51         self._axis_cache_key: Optional[int] = None
52         self._axis_cache: Optional[tuple] = None
53
54         @property
55         def name(self) -> str:
56             return "fusion_hybrid"
57
58         @property
59         def description(self) -> str:
60             return ("Multi-signal fusion: shuttle trajectory (WASB/TrackNet) seeds candidate
rallies, "
61                     "net-crossing Gate A + optional person-occupancy Gate B adjudicate them,
and the "
62                     "AI provider sets semantic boundaries.")
63
64         def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
65             substrate = idx_cfg.get("fusion", {}).get("substrate", "wasb")
66             if substrate == "tracknet":
67                 cfg = TrackNetConfig.from_indexing_cfg(idx_cfg)
68                 return TrackNetRunner(cfg), {
69                     "weights_path": cfg.weights_path,
70                     "queue_length": cfg.queue_length,
71                     "fusion_substrate": "tracknet",
72                 }
73             wcfg = WasbConfig.from_indexing_cfg(idx_cfg)
74             return WasbRunner(wcfg), {"weights_path": wcfg.weights_path, "fusion_substrate":
"wasb"}
75
76         def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
77                                     frame_width: float, video_path: str,
78                                     idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
79             stats = super()._enrich_candidate_stats(cw, points, fps, frame_width,
video_path, idx_cfg)
80             # Gate-A signal ? net-crossing count for this window. Pure, GPU-free, always
persisted
81             # (single service feed crosses ~once; a real rally crosses >=2x). axis/net auto-
estimated
82             # over the whole trajectory by rally_gate (camera-agnostic); reused across
windows.
83             gated = rally_gate.gate_windows(points, [(cw["start"], cw["end"])], fps,
84                                                  min_crossings=0,
estimate=self._axis_estimate(points))
85             stats["net_crossings"] = float(gated[0].crossings) if gated else 0.0
86
87             fcfg = idx_cfg.get("fusion", {})
88             if fcfg.get("cue_flags", {}).get("person", False):
89                 stats.update(self._person_features(cw, points, fps, frame_width, video_path,
fcfg))
90             return stats
91
92         def _axis_estimate(self, points: List[Any]) -> tuple:
93             """Cache rally_gate's (axis, net, span) play-axis estimate per trajectory so
it's
94             computed once per video, not once per candidate window (it depends only on
points)."""
95             key = id(points)

```

```

96         if self._axis_cache_key != key or self._axis_cache is None:
97             self._axis_cache_key = key
98             self._axis_cache = rally_gate.estimate_play_axis(points)
99         est = self._axis_cache
100         assert est is not None
101         return est
102
103     def _person_features(self, cw: Dict[str, Any], points: List[Any], fps: float,
104                         frame_width: float, video_path: str,
105                         fcfg: Dict[str, Any]) -> Dict[str, float]:
106         """GPU path (person cue ON): run the person detector over this window's
ORIGINAL-video
107         frame range and compute the scale/translation-invariant person features. Lazily
builds
108         ONE PersonDetector. ? real-video verification is owner-side; this wiring is
mock-tested."""
109         # Lazy import so the default (person-OFF) path never pulls torch/cv2 through
this module.
110         from backend.pipeline.detectors.person_detector import PersonDetector
111         if self._person is None:
112             self._person = PersonDetector(self.config)
113
114         frame_skip = self.config.get("indexing", {}).get("yolo_frame_skip", 3)
115         # round (not truncate): int() would map e.g. 1.999s*30 to frame 59 not 60 ? a
116         # window-edge off-by-one that drops the boundary frame from the shuttle
alignment.
117         start_f, end_f = round(cw["start"] * fps), round(cw["end"] * fps)
118         det = self._person.detections_per_frame(video_path, start_f, end_f, frame_skip)
119         fw = det["frame_width"] or frame_width
120         fh = det["frame_height"]
121         # If we can't trust the frame dims, every normalised feature would silently
collapse
122         # to -0 ? return an explicit all-zero person-feature row instead of a half-baked
one.
123         if fw <= 0 or fh <= 0:
124             return {k: 0.0 for k in ff.PERSON_FEATURE_NAMES}
125
126         poly = fcfg.get("court_polygon")
127         polygon = [(float(p[0]), float(p[1])) for p in poly] if poly else None
128         net = (fcfg.get("net_axis", "y"), float(fcfg.get("net_position", 0.5)))
129         shuttle_in = [p for p in points if start_f <= p.frame <= end_f]
130         return ff.compute_person_features(det["frames"], shuttle_in, fw, fh,
131                                         court_polygon=polygon, net=net)
132
133     def _select_for_handoff(self, windows: List[Dict[str, Any]],
134                           window_stats: List[Dict[str, Any]],
135                           idx_cfg: Dict[str, Any]) -> List[int]:
136         """Served Gate A/B over the (already-enriched) per-window stats. Default
thresholds
137         (0) keep every window ? identical to the substrate. Persisted candidates are
138         untouched (full superset)."""
139         fcfg = idx_cfg.get("fusion", {})
140         min_crossings = int(fcfg.get("min_crossings", 0))
141         min_players = int(fcfg.get("min_players", 0))
142         person_on = bool(fcfg.get("cue_flags", {}).get("person", False))
143
144         keep: List[int] = []
145         for i, st in enumerate(window_stats):
146             if min_crossings > 0 and st.get("net_crossings", 0.0) < min_crossings:
147                 continue
148             if person_on and min_players > 0 and st.get("players_in_court", 0.0) <
min_players:
149                 continue
150             keep.append(i)
151         return keep

```

```
152
153
154     segmenter_registry.register(FusionSegmenter)
155
```

14. assistant (2026-06-13T19:55:43.672Z)

Now let me search for PersonDetector to understand its interface:

15. user (2026-06-13T19:55:44.825Z)

```
C:/Users/avidu/Projects/badminton-highlight-
indexer/backend/pipeline/detectors/person_detector.py
```

16. user (2026-06-13T19:55:46.332Z)

```
1     """Person-detection rally cue ? torchvision detector + ByteTrack + motion windowing.
2
3     Extracted verbatim from `yolo_hybrid` (fusion P1, `docs/MULTI_SIGNAL_FUSION_PLAN.md`):
4     the **person cue is a swappable producer**, not logic baked into one segmenter. Today
5     `HybridYoloSegmenter` is the only consumer (it delegates Stage-1 here, byte-
identically);
6     next the fusion segmenter consumes the SAME producer over shuttle-seeded windows, and
7     later the learned classifier reads its persisted features. Keeping detection here (not
in
8     the segmenter) is what makes "add a cue = register a producer" true.
9
10    Behaviour is unchanged from the in-lined version ? only the home moved:
11    - `lazy_load_model`           ? load a permissively-licensed torchvision detector
12    - `make_tracker`             ? a fresh supervision ByteTrack per chunk
13    - `detect_and_track`         ? one frame ? confident persons with persistent IDs
14    - `extract_action_windows_from_chunk` ? a chunk ? high-motion candidate windows
15    with the motion stats (`peak/mean_velocity`, `active_frame_count`, `track_id_count`)
16    that feed the candidate `stats` JSON and the learned fusion features.
17
18    The model libs (torchvision, supervision) are imported lazily inside the methods, and
the
19    detector weights load only on first detection (`lazy_load_model`) ? so importing the
20    segmenter registry never loads a detector model, and tests can mock the seams. (torch +
cv2
21    are top-level imports here; CI's import-smoke stubs them, and they're present on the GPU
22    machine.) Licensing: torchvision detectors are BSD + COCO weights; ByteTrack is MIT
23    (ADR-005; ultralytics' AGPL YOLO was dropped).
24    """
25    import logging
26    from typing import Any, Dict, List, Optional, Tuple
27
28    import cv2
29    import torch
30
31    from backend.utils.geometry import get_velocity_normalised
32
33    logger = logging.getLogger(__name__)
34
35    # torchvision detection models are trained on COCO where the "person" category is
36    # class 1 (index 0 is the implicit background). NB: ultralytics YOLO used class 0 ?
37    # this off-by-one is the classic gotcha when swapping detectors. Keep it named.
38    _PERSON_CLASS_ID = 1
39
40    # Permissively-licensed torchvision detectors selectable via config
41    # `indexing.detector_model`. All are BSD-licensed (code + COCO weights).
42    _DETECTOR_FACTORIES = {
43        "fasterrcnn_resnet50_fpn": "fasterrcnn_resnet50_fpn",
44        "fasterrcnn_mobilenet_v3_large_fpn": "fasterrcnn_mobilenet_v3_large_fpn",
```

```

45         "retinanet_resnet50_fpn": "retinanet_resnet50_fpn",
46     }
47     _DEFAULT_DETECTOR = "fasterrcnn_resnet50_fpn"
48
49
50     class PersonDetector:
51         """Stage-1 person detection + tracking + motion-window extraction (a rally cue).
52
53         Owns the torchvision model + per-chunk ByteTrack tracker. Constructed with the run
54         ``config`` (reads ``indexing.{use_gpu, detector_model, detector_score_threshold,
55         min_action_window_duration}``); the model loads lazily on first use.
56         """
57
58         def __init__(self, config: Dict[str, Any]):
59             self.config = config
60             # Dynamic torchvision module (None until lazy_load_model); typed Any so the
61             # `self.model([tensor])` inference call type-checks without a quarantine.
62             self.model: Any = None
63             self.device: Optional[str] = None
64
65         def lazy_load_model(self) -> None:
66             if self.model is not None:
67                 return
68
69             # Imported lazily so the module imports without torchvision present and so
70             # the test suite (which pre-sets self.model) never pulls in model weights.
71             from torchvision.models import detection as tv_detection
72
73             idx_cfg = self.config.get("indexing", {})
74             use_gpu = idx_cfg.get("use_gpu", True)
75             self.device = "cuda" if use_gpu and torch.cuda.is_available() else "cpu"
76             if use_gpu and not torch.cuda.is_available():
77                 logger.warning("use_gpu is True but CUDA is not available. Falling back to
CPU.")
78
79             model_name = idx_cfg.get("detector_model", _DEFAULT_DETECTOR)
80             if model_name not in _DETECTOR_FACTORIES:
81                 logger.warning(f"Unknown detector_model '{model_name}'. Falling back to
{_DEFAULT_DETECTOR}.")
82             model_name = _DEFAULT_DETECTOR
83
84             builder = getattr(tv_detection, _DETECTOR_FACTORIES[model_name])
85             logger.info(f"Loading torchvision detector '{model_name}' on {self.device}...")
86             # weights="DEFAULT" pulls the best available COCO-pretrained checkpoint.
87             self.model = builder(weights="DEFAULT")
88             self.model.eval()
89             self.model.to(self.device)
90
91         def make_tracker(self, fps: float) -> Any:
92             """Create a fresh ByteTrack tracker for one chunk (track IDs are chunk-local).
93
94             Imported lazily so tests can mock this method without supervision installed.
95             """
96             import supervision as sv
97             frame_rate = max(1, int(round(fps)))
98             return sv.ByteTrack(frame_rate=frame_rate)
99
100         def detect_and_track(self, frame: Any, tracker: Any,
101                             score_thresh: float) -> List[Tuple[int, Tuple[float, float,
float, float]]]:
102             """Detect persons in a frame and assign persistent track IDs.
103
104             Returns a list of (track_id, bbox) where bbox is (x1, y1, x2, y2) in pixels.
105             This replaces the old ultralytics ``model.track(...)`` call, which bundled
106             detection AND association: torchvision does the detection, supervision's

```

```

107         ByteTrack does the association.
108         """
109         import supervision as sv
110
111         # cv2 frames are BGR uint8 HxWxC; torchvision wants RGB float [0,1] CxHxW.
112         rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
113         tensor = torch.from_numpy(rgb).permute(2, 0,
114 1).float().div(255.0).to(self.device)
115         with torch.no_grad():
116             outputs = self.model([tensor])
117
118         out = outputs[0]
119         boxes = out["boxes"].detach().cpu().numpy()
120         labels = out["labels"].detach().cpu().numpy()
121         scores = out["scores"].detach().cpu().numpy()
122
123         # Keep only confident person detections.
124         mask = (labels == _PERSON_CLASS_ID) & (scores >= score_thresh)
125         if not mask.any():
126             return []
127
128         # ByteTrack requires confidence to be populated (it filters on it internally).
129         dets = sv.Detections(xyxy=boxes[mask], confidence=scores[mask],
130 class_id=labels[mask])
131         tracked = tracker.update_with_detections(dets)
132
133         result: List[Tuple[int, Tuple[float, float, float, float]]] = []
134         for i in range(len(tracked)):
135             tid = tracked.tracker_id[i]
136             if tid is None:
137                 continue
138             x1, y1, x2, y2 = tracked.xyxy[i]
139             result.append((int(tid), (float(x1), float(y1), float(x2), float(y2))))
140         return result
141
142     def extract_action_windows_from_chunk(self, chunk_path: str, chunk_start: float,
143 velocity_thresh: float, merge_gap: float,
144 frame_skip: int = 3,
145 chunk_index: Optional[int] = None) ->
146 List[Dict[str, Any]]:
147     """
148     Runs person detection + tracking on a video chunk and extracts high-motion
149     action windows. Returns a list of dicts in the full video's timeframe, each
150     with:
151         start, end (seconds), active_frame_count, peak_velocity, mean_velocity,
152         track_id_count, chunk_index.
153
154     Args:

```

17. user (2026-06-13T19:55:48.171Z)

```

150     Args:
151         frame_skip: Process every Nth frame (1 = every frame). Higher values
152             dramatically reduce inference cost with minimal quality loss.
153         chunk_index: Index of the source chunk (recorded on each window for
154 traceability).
155     """
156     cap = cv2.VideoCapture(chunk_path)
157     if not cap.isOpened():
158         logger.error(f"Could not open {chunk_path}")
159         return []
160
161     fps = cap.get(cv2.CAP_PROP_FPS)
162     if fps <= 0:
163         fps = 30.0

```



```

163
164     frame_width = cap.get(cv2.CAP_PROP_FRAME_WIDTH)
165     if frame_width <= 0:
166         frame_width = 1280.0 # Sensible fallback
167
168     # Effective sampling rate after frame-skipping
169     effective_fps = fps / max(frame_skip, 1)
170
171     # A fresh tracker per chunk ? track IDs only need to be consistent within a
172     # chunk (windows are computed per chunk, then merged across chunks by time).
173     tracker = self.make_tracker(effective_fps)
174     score_thresh = self.config.get("indexing", {}).get("detector_score_threshold",
0.5)
175
176     # Per active frame we keep (frame_idx, peak_velocity, set_of_active_track_ids)
177     # so windows can be enriched with motion stats for logging + fine-tuning.
178     active_frames = []
179     frame_idx = 0
180     track_history: Dict[int, Tuple[float, float, float, float]] = {}
181
182     while True:
183         ret, frame = cap.read()
184         if not ret:
185             break
186
187         # Skip frames for performance ? at 30fps, every 3rd frame still
188         # gives ~10 samples/second, plenty for human-scale velocity.
189         if frame_idx % max(frame_skip, 1) != 0:
190             frame_idx += 1
191             continue
192
193         # Detect persons + assign persistent track IDs.
194         detections = self.detect_and_track(frame, tracker, score_thresh)
195
196         frame_peak_velocity = 0.0
197         frame_active_ids = set()
198         current_ids = set()
199
200         for track_id, bbox in detections:
201             current_ids.add(track_id)
202
203             if track_id in track_history:
204                 prev_bbox = track_history[track_id]
205                 velocity = get_velocity_normalised(prev_bbox, bbox, effective_fps,
frame_width)
206
207                 if velocity > velocity_thresh:
208                     frame_peak_velocity = max(frame_peak_velocity, velocity)
209                     frame_active_ids.add(track_id)
210
211                 track_history[track_id] = bbox
212
213         # Prune stale IDs that are no longer visible ? prevents false
214         # velocity spikes when the tracker reassigns an old ID.
215         track_history = {k: v for k, v in track_history.items() if k in current_ids}
216
217         if frame_active_ids:
218             active_frames.append((frame_idx, frame_peak_velocity, frame_active_ids))
219
220         frame_idx += 1
221
222     cap.release()
223
224     # Group active frames into continuous windows
225     if not active_frames:
226         return []

```

```

226
227     max_gap_frames = int(merge_gap * fps)
228
229     def _finalize(group: List[Tuple[int, float, set]]) -> Dict[str, Any]:
230         velocities = [v for _, v, _ in group]
231         track_ids = set().union(*[ids for _, _, ids in group])
232         return {
233             "start": group[0][0] / fps + chunk_start,
234             "end": group[-1][0] / fps + chunk_start,
235             "active_frame_count": len(group),
236             "peak_velocity": max(velocities),
237             "mean_velocity": sum(velocities) / len(velocities),
238             "track_id_count": len(track_ids),
239             "chunk_index": chunk_index,
240         }
241
242     windows = []
243     group = [active_frames[0]]
244     for af in active_frames[1:]:
245         if af[0] - group[-1][0] <= max_gap_frames:
246             group.append(af)
247         else:
248             windows.append(_finalize(group))
249             group = [af]
250     windows.append(_finalize(group))
251
252     # Filter out extremely short action windows
253     min_window_duration = self.config.get("indexing",
254     {}).get("min_action_window_duration", 2.0)
255     windows = [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]
256
257     return windows
258
259     def detections_per_frame(self, video_path: str, start_frame: int, end_frame: int,
260                             frame_skip: int = 3) -> Dict[str, Any]:
261         """Per-frame person detections over ``[start_frame, end_frame]`` of the ORIGINAL
262         video.
263
264         Used by the fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2). Unlike
265         ``extract_action_windows_from_chunk`` (which detects ITS OWN windows and
266         discards the
267         per-frame boxes), this returns the raw per-frame detections so the fusion
268         features can
269         align them with the shuttle trajectory. It reads the source by **absolute frame
270         index**
271         (seek + sequential read), NOT a re-extracted clip, so the returned frame indices
272         match
273         the WASB trajectory's ``.frame`` exactly ? that alignment is what
274         ``fusion_features.shuttle_player_colocation`` / ``two_sided_motion`` rely on.
275
276         Returns ``{"frames": {frame_idx: [(track_id, bbox), ...]}, "fps", "frame_width",
277         "frame_height"}``. A fresh window-local ByteTrack tracker is used. ? cv2 frame-
278         seek
279         accuracy varies by codec (it lands near the requested keyframe then decodes
280         forward);
281         treat the indices as ±a few frames on real footage ? fine for window-level
282         features.
283
284         """
285         self.lazy_load_model()
286         cap = cv2.VideoCapture(video_path)
287         if not cap.isOpened():
288             logger.error(f"Could not open {video_path}")
289             return {"frames": {}, "fps": 30.0, "frame_width": 0.0, "frame_height": 0.0}
290
291         fps = cap.get(cv2.CAP_PROP_FPS)

```

```

282         if fps <= 0:
283             fps = 30.0
284         frame_width = cap.get(cv2.CAP_PROP_FRAME_WIDTH)
285         frame_height = cap.get(cv2.CAP_PROP_FRAME_HEIGHT)
286         score_thresh = self.config.get("indexing", {}).get("detector_score_threshold",
0.5)
287         step = max(frame_skip, 1)
288         tracker = self.make_tracker(fps / step)
289
290         start_frame = max(0, int(start_frame))
291         frames: Dict[int, List[Tuple[int, Tuple[float, float, float, float]]]] = {}
292         cap.set(cv2.CAP_PROP_POS_FRAMES, start_frame)
293         frame_idx = start_frame
294         while frame_idx <= end_frame:
295             ret, frame = cap.read()
296             if not ret:
297                 break
298             if (frame_idx - start_frame) % step == 0:
299                 frames[frame_idx] = self.detect_and_track(frame, tracker, score_thresh)
300                 frame_idx += 1
301         cap.release()
302         return {"frames": frames, "fps": fps,
303               "frame_width": float(frame_width), "frame_height": float(frame_height)}
304

```

18. assistant (2026-06-13T19:55:49.596Z)

Now let me check what metrics and other evaluation utilities are available:

19. user (2026-06-13T19:55:50.115Z)

```

1     """Pure temporal interval-matching metrics for rally evaluation.
2
3     No I/O, no DB, no global state ? every function here is deterministic and unit
4     tested. An "interval" is a `(start, end)` tuple of floats in seconds with
5     start < end. We score a list of *predicted* rally intervals against a list of
6     *ground-truth* intervals via temporal IoU and one-to-one greedy matching.
7     """
8
9     from dataclasses import dataclass, field
10    from typing import List, Tuple, Optional
11
12    Interval = Tuple[float, float]
13
14
15    def interval_iou(a: Interval, b: Interval) -> float:
16        """Temporal intersection-over-union of two intervals.
17
18        Returns 0.0 when they don't overlap (or either is degenerate). IoU is
19        overlap / union, both measured as durations on the timeline.
20        """
21        a_start, a_end = a
22        b_start, b_end = b
23        if a_end <= a_start or b_end <= b_start:
24            return 0.0
25        inter = max(0.0, min(a_end, b_end) - max(a_start, b_start))
26        if inter <= 0.0:
27            return 0.0
28        union = (a_end - a_start) + (b_end - b_start) - inter
29        return inter / union if union > 0 else 0.0
30
31
32    @dataclass
33    class Match:
34        """One matched predicted?ground-truth pair."""

```

```

35     pred_index: int
36     gt_index: int
37     iou: float
38
39
40 @dataclass
41 class MatchResult:
42     """Outcome of matching predictions to ground truth at a given IoU threshold."""
43     matches: List[Match] = field(default_factory=list)
44     unmatched_pred_indices: List[int] = field(default_factory=list) # false positives
45     unmatched_gt_indices: List[int] = field(default_factory=list)   # false negatives
(misses)
46
47
48 def match_intervals(preds: List[Interval], gts: List[Interval],
49                    iou_threshold: float = 0.5) -> MatchResult:
50     """Greedy one-to-one matching of predictions to ground truth by descending IoU.
51
52     Every candidate (pred, gt) pair with IoU >= threshold is considered; pairs
53     are claimed highest-IoU first, and each prediction/GT can be used once. This
54     keeps a single prediction from being credited for two ground-truth rallies
55     (and vice versa).
56     """
57     candidates = []
58     for pi, p in enumerate(preds):
59         for gi, g in enumerate(gts):
60             iou = interval_iou(p, g)
61             # Require real overlap: at iou_threshold=0.0 a plain `iou >= threshold`
would
62             # match completely disjoint intervals (IoU 0.0), fabricating perfect scores.
63             if iou > 0.0 and iou >= iou_threshold:
64                 candidates.append((iou, pi, gi))
65     # Highest IoU first; ties broken by indices for determinism.
66     candidates.sort(key=lambda c: (-c[0], c[1], c[2]))
67
68     used_pred, used_gt = set(), set()
69     matches: List[Match] = []
70     for iou, pi, gi in candidates:
71         if pi in used_pred or gi in used_gt:
72             continue
73         used_pred.add(pi)
74         used_gt.add(gi)
75         matches.append(Match(pred_index=pi, gt_index=gi, iou=iou))
76
77     unmatched_pred = [i for i in range(len(preds)) if i not in used_pred]
78     unmatched_gt = [i for i in range(len(gts)) if i not in used_gt]
79     return MatchResult(matches=matches,
80                        unmatched_pred_indices=unmatched_pred,
81                        unmatched_gt_indices=unmatched_gt)
82
83
84 @dataclass
85 class Score:
86     """Aggregate metrics for one (predictions vs ground-truth) evaluation."""
87     iou_threshold: float
88     num_pred: int
89     num_gt: int
90     true_positives: int
91     false_positives: int
92     false_negatives: int
93     precision: float
94     recall: float
95     f1: float
96     mean_iou: float # over matched pairs (0.0 if none)
97     mean_start_error: float # mean |pred_start - gt_start| over matches, seconds

```

```
98         mean_end_error: float          # mean |pred_end - gt_end| over matches, seconds
99
100         def as_dict(self) -> dict:
```

20. assistant (2026-06-13T19:55:51.632Z)

Perfect. Now let me look at one more crucial piece - how VideoCandidates flows through to understand the seams:

21. user (2026-06-13T19:55:52.271Z)

```
Found 2 files
tests\test_experiment.py
backend\eval\experiment.py
```

22. assistant (2026-06-13T19:55:53.707Z)

Let me check the test to see how it's used:

23. user (2026-06-13T19:55:54.246Z)

```
1      """Tests for the experiment harness (backend.eval.experiment).
2
3      Pure/offline ? synthetic candidate rows, no GPU/WSL. The load-bearing test is the
4      **no-regression invariant** (`test_disabled_cues_byte_identical_to_control`): a variant
5      that merely carries a disabled cue must predict exactly what control predicts. That is
the
6      guarantee that lets new cues be merged freely (`EXPERIMENT_HARNESS.md` §6).
7      """
8      import json
9
10     import pytest
11
12     from backend.eval.classifier import LogisticRegression
13     from backend.eval.experiment import (
14         CONTROL,
15         ExperimentError,
16         Variant,
17         VideoCandidates,
18         compare,
19         compare_unlabeled,
20         evaluate_variant,
21         load_video_candidates,
22         predict,
23         record_experiment,
24     )
25     from backend.infrastructure.database import Database
26
27
28     # A video whose third candidate is a held-shuttle false fire (net_crossings=0): it does
29     # not overlap any golden rally, so it is a negative by construction.
30     def _held_shuttle_video(name="v", gts=None):
31         return VideoCandidates(
32             name=name,
33             intervals=[(0.0, 10.0), (20.0, 30.0), (12.0, 13.0)],
34             features={"net_crossings": [4.0, 3.0, 0.0]},
35             gts=gts if gts is not None else [(0.0, 10.0), (20.0, 30.0)],
36         )
37
38
39     # Three videos where feature "x" perfectly separates rallies (x=5) from junk (x=0),
40     # so an honest LOVO learned variant should recover F1?1.0 on each held-out video.
41     def _separable_videos():
42         vids = []
```

```

43     for k in range(3):
44         base = 100.0 * k
45         vids.append(VideoCandidates(
46             name=f"sep{k}",
47             intervals=[(base + 0, base + 10), (base + 20, base + 30),
48                       (base + 40, base + 41), (base + 50, base + 51)],
49             features={"x": [5.0, 5.0, 0.0, 0.0]},
50             gts=[(base + 0, base + 10), (base + 20, base + 30)],
51         ))
52     return vids
53
54
55     # ----- Variant
56
57     def test_variant_roundtrip_and_hash_stability():
58         v = Variant(name="gateA", min_crossings=2, cue_flags={"person": False})
59         assert Variant.from_dict(v.to_dict()) == v
60         assert v.spec_hash() == v.spec_hash() # stable
61         assert CONTROL.spec_hash() == Variant(name="control").spec_hash()
62         assert v.spec_hash() != CONTROL.spec_hash() # different recipe ?
different hash
63
64
65     def test_from_dict_ignores_unknown_keys():
66         v = Variant.from_dict({"name": "x", "min_crossings": 1, "bogus": 99})
67         assert v.name == "x" and v.min_crossings == 1
68
69
70     def test_is_learned():
71         assert not CONTROL.is_learned()
72         assert Variant(name="r", feature_names=["x"]).is_learned()
73         assert Variant(name="p", classifier_model="model.json").is_learned()
74
75
76     # ----- the no-regression invariant
77
78     def test_disabled_cues_byte_identical_to_control():
79         """A variant carrying only DISABLED cues must equal control, window-for-window."""
80         vc = _held_shuttle_video()
81         disabled = Variant(name="fusion_off", cue_flags={"person": False, "audio": False},
82                           min_crossings=0, min_players=0)
83         assert predict(disabled, vc) == predict(CONTROL, vc)
84
85
86     def test_control_is_merged_candidates():
87         vc = VideoCandidates(name="v", intervals=[(0.0, 10.0), (10.5, 12.0), (40.0, 50.0)])
88         # merge_gap=1.0 stitches the 0.5s gap between the first two; the 28s gap stays
split.
89         assert predict(CONTROL, vc) == [(0.0, 12.0), (40.0, 50.0)]
90
91
92     # ----- decision gates
93
94     def test_gate_a_drops_low_crossing_window():
95         vc = _held_shuttle_video()
96         gate_a = Variant(name="gateA", min_crossings=2)
97         preds = predict(gate_a, vc)
98         assert (12.0, 13.0) not in preds # held-shuttle fire
removed
99         assert preds == [(0.0, 10.0), (20.0, 30.0)]
100         assert preds != predict(CONTROL, vc) # the gate actually
changed output
101
102
103     def test_gate_b_filters_on_players():

```

```

104     vc = VideoCandidates(
105         name="v", intervals=[(0.0, 10.0), (20.0, 30.0)],
106         features={"players_in_court": [2.0, 1.0]})
107     preds = predict(Variant(name="gateB", min_players=2), vc)
108     assert preds == [(0.0, 10.0)] # the 1-player window is
dropped
109
110
111     def test_gate_on_absent_feature_defaults_to_zero(caplog):
112         vc = VideoCandidates(name="v", intervals=[(0.0, 10.0)]) # no net_crossings
persisted
113         preds = predict(Variant(name="gateA", min_crossings=2), vc)
114         assert preds == [] # absent feature ? 0 ?
fails gate
115
116
117     # ----- evaluate_variant
118
119     def test_evaluate_static_variant_scores_vs_gts():
120         res = evaluate_variant([_held_shuttle_video()], CONTROL, iou=0.5)
121         pv = res["per_video"][0]
122         # control keeps the false fire: 2 TP, 1 FP ? precision 2/3, recall 1.0
123         assert pv["recall"] == pytest.approx(1.0)
124         assert pv["precision"] == pytest.approx(2 / 3)
125         assert res["aggregate"]["recall"] == pytest.approx(1.0)
126         assert not res["honest_lovo"]
127
128
129     def test_evaluate_learned_lovo_recovers_separable_signal():
130         variant = Variant(name="learned", feature_names=["x"], threshold=0.5)
131         res = evaluate_variant(_separable_videos(), variant, iou=0.5, honest=True)
132         assert res["honest_lovo"] is True
133         assert res["aggregate"]["f1"] == pytest.approx(1.0) # held-out F1 perfect on
separable data
134
135
136     def test_evaluate_learned_lovo_needs_two_videos():
137         variant = Variant(name="learned", feature_names=["x"])
138         with pytest.raises(ExperimentError):
139             evaluate_variant(_separable_videos()[:1], variant, honest=True)
140
141
142     def test_evaluate_requires_labels():
143         with pytest.raises(ExperimentError):
144             evaluate_variant([VideoCandidates(name="v", intervals=[(0.0, 1.0)], gts=None)],
CONTROL)
145
146
147     # ----- compare
148
149     def test_compare_gate_a_improves_precision():
150         videos = [_held_shuttle_video()]
151         res = compare(videos, CONTROL, Variant(name="gateA", min_crossings=2), iou=0.5)
152         assert res["variant_b"]["aggregate"]["precision"] == pytest.approx(1.0)
153         assert res["delta"]["precision"] > 0 # B (gate) beats A
(control) on precision
154         assert res["delta"]["f1"] > 0
155         assert res["label_set_hash"] # fingerprint present
156         assert res["num_videos"] == 1
157         assert len(res["per_video_delta"]) == 1
158
159
160     def test_compare_needs_a_video():
161         with pytest.raises(ExperimentError):
162             compare([], CONTROL, CONTROL)

```

```

163
164
165 # ----- compare_unlabeled
166
167 def test_compare_unlabeled_partitions_agreement():
168     vc = _held_shuttle_video(gts=None) # new footage, no labels
169     res = compare_unlabeled(vc, CONTROL, Variant(name="gateA", min_crossings=2))
170     assert res["num_a"] == 3 and res["num_b"] == 2
171     assert res["num_agreed"] == 2
172     assert res["a_only"] == [(12.0, 13.0)] # A fired, B suppressed
173     assert res["b_only"] == [] # B is a subset of A here
174     assert res["mean_agree_iou"] == pytest.approx(1.0)
175
176
177 def test_compare_unlabeled_learned_variant_needs_pinned_model():
178     vc = VideoCandidates(name="new", intervals=[(0.0, 10.0)], features={"x": [5.0]})
179     learned_no_model = Variant(name="learned", feature_names=["x"])
180     with pytest.raises(ExperimentError):
181         compare_unlabeled(vc, CONTROL, learned_no_model)
182
183
184 def test_compare_unlabeled_with_pinned_model(tmp_path):
185     # Train + persist a model, then use it as a pinned variant on an unlabeled video.
186     model = LogisticRegression().fit([[5.0], [5.0], [0.0], [0.0]], [1, 1, 0, 0], ["x"])
187     model_path = str(tmp_path / "m.json")
188     model.save(model_path)
189     vc = VideoCandidates(name="new", intervals=[(0.0, 10.0), (40.0, 41.0)],
190                           features={"x": [5.0, 0.0]})
191     pinned = Variant(name="learned", feature_names=["x"], classifier_model=model_path)
192     res = compare_unlabeled(vc, CONTROL, pinned)
193     assert res["num_a"] == 2 # control keeps both
194     assert res["num_b"] == 1 # model drops the x=0
195
196 window
195     assert res["a_only"] == [(40.0, 41.0)]
196
197
198 # ----- record_experiment
199
200 def test_record_experiment_appends(tmp_path):

```

24. assistant (2026-06-13T19:55:55.917Z)

Excellent. Now let me check if there are existing integration patterns or scripts that show how to orchestrate the full pipeline:

25. user (2026-06-13T19:55:56.650Z)

```

C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/distill_local.py
C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/training.py
C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/__pycache__/distill_local.cpython-313.pyc
C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/__pycache__/training.cpython-313.pyc
C:/Users/avidu/Projects/badminton-highlight-indexer/tests/test_distill_local.py
C:/Users/avidu/Projects/badminton-highlight-indexer/tests/test_eval_training.py
C:/Users/avidu/Projects/badminton-highlight-indexer/tests/test_training_gen0.py
C:/Users/avidu/Projects/badminton-highlight-indexer/tests/__pycache__/test_distill_local.cpython-313-pytest-9.0.3.pyc
C:/Users/avidu/Projects/badminton-highlight-indexer/tests/__pycache__/test_distill_local.cpython-313.pyc
C:/Users/avidu/Projects/badminton-highlight-indexer/tests/__pycache__/test_eval_training.cpython-313-pytest-9.0.3.pyc
C:/Users/avidu/Projects/badminton-highlight-indexer/tests/__pycache__/test_eval_training.cpython-313.pyc
C:/Users/avidu/Projects/badminton-highlight-indexer/

```


indexer/tests/___pycache___/test_training_gen0.cpython-313-pytest-9.0.3.pyc
C:/Users/avidu/Projects/badminton-highlight-indexer/training
C:/Users/avidu/Projects/badminton-highlight-indexer/training/requirements-train.txt

26. assistant (2026-06-13T19:55:57.958Z)

Let me now create a comprehensive summary by also checking rally_gate.py to understand Gate A:

27. user (2026-06-13T19:55:58.282Z)

```
1      """Rally-start gate: suppress between-point false fires using shuttle net-crossings.
2
3      WASB (shuttle-only) over-detects: when a player flicks/tosses the shuttle back to
4      the opponent for service after losing a point, that single trajectory looks like a
5      rally. The discriminator is structural, not duration-based: a real rally has the
6      shuttle crossing the net MULTIPLE times (each shot lands on the other side); a
7      single service feed crosses ~once.
8
9      This module is camera-agnostic and needs NO new model ? it works on the trajectory
10     we already produce (Frame,Visibility,X,Y). It estimates the play axis (the image
11     axis with the larger shuttle spread) and the net position (median shuttle coord on
12     that axis ? the shuttle clusters at the two ends and crosses the net region often,
13     so the median sits near the net), then counts sign changes of (coord - net) across
14     the visible points inside a candidate window, with a deadband to ignore jitter.
15
16     Gate A in the over-fire fix spectrum (B = player-stance state machine, future).
17     Pure functions only ? no I/O, unit-tested; intended as a post-filter on the
18     windows from trajectory_to_action_windows, or to fold into it later.
19     """
20     from __future__ import annotations
21
22     from dataclasses import dataclass
23     from typing import List, Optional, Sequence, Tuple
24
25     Interval = Tuple[float, float]
26
27
28     @dataclass
29     class GatedWindow:
30         start: float
31         end: float
32         crossings: int
33         visible_points: int
34         kept: bool
35
36
37     def _spread(vals: Sequence[float]) -> float:
38         """Robust spread = p95 - p5 (ignores a few outlier detections)."""
39         if len(vals) < 2:
40             return 0.0
41         s = sorted(vals)
42         lo = s[max(0, int(0.05 * (len(s) - 1)))]
43         hi = s[min(len(s) - 1, int(0.95 * (len(s) - 1)))]
44         return hi - lo
45
46
47     def _median(vals: Sequence[float]) -> float:
48         if not vals:
49             return 0.0
50         s = sorted(vals)
51         n = len(s)
52         return s[n // 2] if n % 2 else 0.5 * (s[n // 2 - 1] + s[n // 2])
53
54
55     def estimate_play_axis(points) -> Tuple[str, float, float]:
```

```

56         """Return (axis 'x'/'y', net_value, span) from visible points.
57
58         Play axis = the image axis along which the shuttle travels most (larger spread).
59         For a baseline camera that's Y (players top/bottom); for a side camera, X.
60         """
61         xs = [p.x for p in points if p.visible]
62         ys = [p.y for p in points if p.visible]
63         sx, sy = _spread(xs), _spread(ys)
64         if sx >= sy:
65             return "x", _median(xs), sx
66         return "y", _median(ys), sy
67
68
69     def count_net_crossings(window_points, axis: str, net: float, deadband: float) -> int:
70         """Count sign changes of (coord - net) across visible points, ignoring the
71         deadband zone near the net (so jitter at the net doesn't inflate the count)."""
72         crossings = 0
73         last_side = None
74         for p in window_points:
75             if not p.visible:
76                 continue
77             v = (p.x if axis == "x" else p.y) - net
78             if abs(v) < deadband:
79                 continue
80             side = v > 0

```

28. assistant (2026-06-13T19:55:59.683Z)

Perfect. Now I have enough context. Let me compile the findings into a precise report.

29. user (2026-06-13T19:56:47.605Z)

Structured output provided successfully

30. assistant (2026-06-13T19:56:55.317Z)

Summary

I've mapped the learned-LOVO surface for P3 fusion (shuttle-only vs shuttle+person F1 on 6 golden videos). Here are the exact seams and measurement strategy:

****The 10 Features: 5 Shuttle + 5 Person****

```

**Shuttle** (in `distill_local.FEATURE_NAMES`, line 40):
- `duration`, `peak_velocity`, `mean_velocity`, `active_ratio`, `net_crossings`

**Person** (in `fusion_features.PERSON_FEATURE_NAMES`, lines 194?197):
- `players_in_court`, `two_sided_motion`, `mean_player_motion`, `in_court_ratio`,
`shuttle_player_colocation`

```

****Build Labeled Fusion Candidates****

1. Call `experiment.load\_video\_candidates(db, video\_id, feature\_names=[all 10], gts=golden\_intervals)` ? `VideoCandidates` with both feature sets
2. `training.label\_candidates(intervals, gts, iou=0.5)` ? binary labels (same IoU matcher as evaluator)
3. Result: per-candidate feature rows + IoU-matched labels

****Measure Held-Out LOVO Lift****

```

Call `experiment.compare(fusion_videos, shuttle_variant, fusion_variant, iou=0.5, honest=True)`:
- Trains shuttle-only (5 features) and fusion (10 features) per LOVO fold
- Reports per-video F1 deltas + aggregate ?F1 (pct improvement included)

```

- Label-set fingerprint guards against measuring against different labels

****Cleanest Seam: Use `experiment.compare()` Directly****

No new machinery needed. The `Variant.feature_names`` field (line 111) was designed for this?plug in two recipes (different feature sets), `compare()` trains both per-fold, reports deltas. `evaluate_variant()` (lines 245?300) implements honest LOVO; the entire LOVO loop is there: per-fold training on the other N?1 videos ? held-out prediction ? IoU matching ? F1 score ? aggregate.

All building blocks (FusionSegmenter, PersonDetector.detections_per_frame, fusion_features, load_video_candidates, evaluate_variant, compare) are shipped on master. A 50-line driver script calling `compare()` with the two Variant recipes is sufficient.